

A Decomposition Layer for Scaling Beyond the Atom Wall

Block / tree-decomposition compilation of graph coloring to neutral-atom hardware

Research Draft — companion to the main paper

May 29, 2026

Abstract

The gadgetized reduction of k -coloring to unit-disk maximum-weight independent set (MWIS) is exact, but its atom count grows as $O((nk)^2)$, so a few-hundred-atom device caps the original graph at a handful of vertices. This note adds a *decomposition layer* in front of the reduction. Instead of compiling the whole graph at once, we split it along small separators, encode one piece at a time with its boundary fixed by precoloring constraints, solve each piece on the device, and stitch the partial colorings classically. The peak hardware size then depends on the *width* w of the decomposition — the size of the largest piece — rather than on n :

$$|V(A)|_{\text{peak}} = O((wk)^2).$$

For graph classes of bounded width this is constant in n , so arbitrarily large graphs become feasible; for general graphs it trades the device-size wall for a classical cost of $O(k^b)$ device solves per piece, where b is the separator size. We prove correctness for clique-separator decomposition (where stitching is free) and state the general tree-decomposition dynamic program, give the atom-count theorem, and finish with fully worked examples.

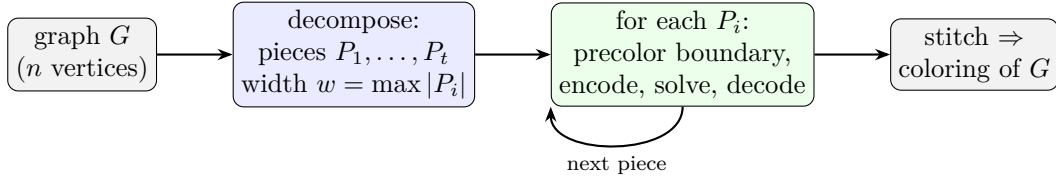
Contents

1	Idea: decouple device size from n	2
2	Preliminaries: separators, blocks, and width	2
3	Clique-separator decomposition: free stitching	2
4	Per-piece encoding by boundary precoloring	3
5	The general case: tree-decomposition dynamic program	4
6	The decomposition-aware pipeline	4
7	The scaling result	5
8	Worked examples	6
8.1	Example A — the bowtie: a single cut vertex	6
8.2	Example B — a chain of triangles: $n \rightarrow \infty$ at constant device size	7
8.3	Example C — a non-clique separator: paying the k^b price	7

1 Idea: decouple device size from n

Recall from the main paper that compiling a coloring instance (G, k) produces a unit-disk MWIS instance A with $nk \leq |V(A)| = O((nk)^2)$ atoms, and that the binding hardware constraint is the atom count $N_{\max}$. The quadratic growth is the wall.

The decomposition layer rests on a classical fact: *coloring is local*. Every constraint of G is a single edge, and edges do not span the whole graph — they live inside small neighborhoods. If we can carve G into overlapping pieces so that every edge sits inside some piece, we can color the pieces one at a time and glue the results, provided the pieces agree on the vertices they share. The device never holds more than one piece, so its size is governed by the *largest piece*, not by G .



The peak device size is $O((wk)^2)$ with w the largest piece, so the entire research question “how large a graph can we handle?” is re-parameterised from n to w .

2 Preliminaries: separators, blocks, and width

Definition 2.1 (Separator). A vertex set $S \subseteq V$ is a *separator* of a connected graph G if $G \setminus S$ has at least two connected components. S is a *clique separator* if $G[S]$ is complete.

Definition 2.2 (Pieces of a separator). Let S separate G into components $C_1, \dots, C_p$ of $G \setminus S$. The *pieces* are the induced subgraphs $G_j = G[S \cup C_j]$. Every piece contains all of S ; the pieces overlap exactly on S , and every edge of G lies inside some piece (an edge is either within S , or has an endpoint in some C_j and hence both endpoints in G_j).

Definition 2.3 (Tree decomposition and width [4]). A *tree decomposition* of G is a tree T whose nodes carry vertex sets (*bags*) $B_x \subseteq V$ such that (i) every vertex lies in some bag, (ii) for every edge $\{u, v\}$ some bag contains both u and v , and (iii) for every vertex v the bags containing v form a connected subtree. The *width* is $\max_x |B_x| - 1$, and the *treewidth* $\text{tw}(G)$ is the minimum width over all tree decompositions.

Throughout, the *decomposition width* w is the size of the largest piece (clique-separator decomposition) or the largest bag (tree decomposition); these are the quantities that will replace n in the atom budget.

3 Clique-separator decomposition: free stitching

The cleanest case is when G can be split along clique separators down to *atoms* (pieces with no further clique separator), as in the classical decomposition of Dirac and Tarjan [3, 5].

Theorem 3.1 (Clique separators reduce coloring to pieces). *Let S be a clique separator of G with pieces $G_1, \dots, G_p$. Then G is k -colorable if and only if every piece G_j is k -colorable. Consequently, by recursion, $\chi(G) = \max_j \chi(\text{atom}_j)$ over the atoms of the clique-separator decomposition.*

Proof. ($\Rightarrow$) A proper k -coloring of G restricts to a proper k -coloring of each induced subgraph G_j .

($\Leftarrow$) We glue two pieces at a time; recursion over the decomposition tree finishes the argument. Suppose G_1 and G_2 share the clique separator S and each has a proper k -coloring, c_1 and c_2 . Because S is a clique, c_1 and c_2 each assign $|S|$ *distinct* colors to S ; both are injections $S \rightarrow \{1, \dots, k\}$, so there is a permutation π of $\{1, \dots, k\}$ with $\pi \circ (c_2|_S) = c_1|_S$ (match the two equal-size color images and extend π arbitrarily on the remaining $k - |S|$ colors). Since recoloring by a permutation preserves properness, $\pi \circ c_2$ is a proper k -coloring of G_2 that *agrees with c_1 on S* . The two now-consistent colorings combine into a proper coloring of $G_1 \cup G_2$: every edge lies in G_1 or G_2 and is properly colored there, and the shared vertices S carry one agreed color. (We use $k \geq |S|$, which holds because $k \geq \chi(G_2) \geq \omega(G_2) \geq |S|$.) $\square$

Remark 3.2 (Why clique separators are “free”). The permutation argument needs *no enumeration*: any two proper colorings of a clique differ only by relabeling colors, so we can always realign a freshly solved piece to its already-colored neighbor in $O(k)$ time. There is no $k^{|S|}$ branching. This is exactly why clique-separator decomposition is the preferred front-end whenever it applies.

4 Per-piece encoding by boundary precoloring

When we solve a piece G_i , the vertices it shares with already-solved neighbors — its *boundary* $B \subseteq V(G_i)$ — already have fixed colors $\beta : B \rightarrow \{1, \dots, k\}$. We bake β into the Stage-1 reduction.

Definition 4.1 (Boundary-precolored color-choice graph). Given a piece $G_i = (V_i, E_i)$, a boundary $B \subseteq V_i$, and a proper partial coloring $\beta : B \rightarrow \{1, \dots, k\}$, define $G'_i(\beta)$ as the color-choice graph of (G_i, k) with one change: each boundary vertex $v \in B$ keeps *only* its single slot $(v, \beta(v))$ (its other $k - 1$ slots and its choice clique are deleted). Interior vertices $v \in V_i \setminus B$ keep all k slots and their choice clique as usual. Conflict edges are kept among all surviving slots.

This is precisely the effect of the language’s **precolor** restriction applied to the boundary, and it shrinks the encoding: the logical-atom count drops to

$$|V(G'_i(\beta))| = k(|V_i| - |B|) + |B| \leq k|V_i|.$$

Proposition 4.2 (Exactness of the precolored encoding). $\alpha(G'_i(\beta)) = |V_i|$ if and only if G_i has a proper k -coloring extending β . Moreover, every independent set of size $|V_i|$ decodes (one chosen slot per vertex) to such an extension.

Proof. This is the Stage-1 exactness theorem of the main paper with some slots pinned. Any independent set takes at most one slot per vertex (the surviving choice cliques on interior vertices; a single available slot on boundary vertices), so its size is at most $|V_i|$, with equality iff exactly one slot per vertex is chosen. The boundary slots $(v, \beta(v))$ are forced choices; they are mutually non-adjacent because β is proper, so they never block each other. A size- $|V_i|$ set therefore picks β on B and one color per interior vertex, and the conflict edges guarantee adjacent vertices differ — i.e. a proper coloring extending β . The converse maps such a coloring to the corresponding slots, which form an independent set of size $|V_i|$. $\square$

Each piece is then gadgetized and solved exactly as in the main paper; only the input graph $G'_i(\beta)$ is smaller and partly pinned.

5 The general case: tree-decomposition dynamic program

Not every graph has helpful clique separators. The general tool is a tree decomposition of width $w = \text{tw}(G)$, processed by the standard dynamic program for coloring on bounded treewidth [1]. Here the separators (adjacent-bag intersections) need not be cliques, so we cannot realign by a single permutation; instead we enumerate the compatible boundary colorings.

Definition 5.1 (Boundary table). For a bag B_x with separator $S_x = B_x \cap B_{\text{parent}(x)}$, the *boundary table* $\mathcal{T}_x$ is the set of proper colorings β of S_x that extend to a proper coloring of the subgraph induced by the subtree rooted at x . There are at most $k^{|S_x|} \leq k^{w+1}$ candidate β .

Proposition 5.2 (Per-bag device subproblem). *Processing the tree from the leaves up, $\beta \in \mathcal{T}_x$ if and only if, for the colorings of the child separators already found compatible, the bag B_x with its separator precolored by β admits a proper completion. Each such test is one instance of Proposition 4.2 on $G[B_x]$, i.e. one device solve of an MWIS instance with $\leq wk$ logical atoms. A nonempty table at the root certifies k -colorability, and a top-down pass selects one compatible β per bag, yielding a global proper coloring.*

This is the classical FPT-in-treewidth algorithm with each local feasibility test delegated to the neutral-atom MWIS solver. The number of device solves is $O(n k^{w+1})$ in the worst case — the price of general (non-clique) separators — while the *peak device size* stays $O((wk)^2)$.

6 The decomposition-aware pipeline

Input: graph $G = (V, E)$, palette k , hardware budget $N_{\max}$.

Output: proper k -coloring of G , or “not k -colorable”.

1. **Preprocess:** iteratively remove vertices of degree $< k$ (the k -core); they are colored greedily at the very end. *// shrinks G*
 2. **Decompose:** compute a clique-separator decomposition into atoms [5]; for atoms still too large, refine with a tree decomposition of width w [2]. Let $P_1, \dots, P_t$ be the pieces, ordered by a rooted decomposition tree. *// $w = \max_i |P_i|$*
 3. **Check budget:** require $O((wk)^2) \leq N_{\max}$; else refine further or report infeasible. *// peak size depends on w , not n*
 4. **For each piece P_i in tree order:**
 - (a) read the boundary coloring β from already-solved neighbors (clique separator: a single β ; general: iterate over the boundary table $\mathcal{T}$);
 - (b) build $G'_i(\beta)$ (Def. 4.1), gadgetize to a UDG MWIS instance, solve on the device, decode (Prop. 4.2);
 - (c) if no size- $|P_i|$ set exists for any β , report “not k -colorable”; else record the piece’s coloring (clique case: realign by the permutation of Thm. 3.1).
 5. **Stitch:** the recorded colorings agree on shared vertices by construction; output their union, then color the peeled degree- $< k$ vertices.
-

Algorithm 1: Decomposition-aware coloring-to-neutral-atom compiler

Theorem 6.1 (Soundness of stitching). *If Algorithm 1 records, for every piece P_i , a proper k -coloring of P_i such that any two pieces agree on their shared vertices, then the union is a proper k -coloring of G .*

Proof. Take any edge $\{u, v\} \in E$. Because every edge lies inside some piece (clique case) or some bag (tree-decomposition case), there is a piece P_i with $u, v \in P_i$. The recorded coloring of P_i is proper, so $c(u) \neq c(v)$. The assignment is well defined because pieces agree on shared vertices. Hence every edge is properly colored. The peeled vertices have degree $< k$ at removal time, so each has a free color when reinserted in reverse order. $\square$

7 The scaling result

Theorem 7.1 (Peak atom count). *Let G admit a decomposition of width w into t pieces with maximum separator size b . Algorithm 1 solves k -coloring with*

$$|V(A)|_{\text{peak}} = O((wk)^2) \text{ atoms on the device at any one time,}$$

using at most t device solves when all separators are cliques (Thm. 3.1), and $O(tk^b)$ device solves in general (Prop. 5.2). The classical work outside the device is $\text{poly}(n)$ per solve.

Proof. At any moment the device holds the gadgetization of a single $G'_i(\beta)$, which has at most wk logical atoms (Def. 4.1) and hence $O((wk)^2)$ atoms after gadgetization (main-paper atom bound). The solve counts are those of Theorem 3.1 (one β per piece) and Proposition 5.2 ($\leq k^b$ boundary colorings per piece). Building each $G'_i(\beta)$, gadgetizing, decoding, and stitching are all polynomial. $\square$

The wall has moved from n to w . Three regimes make the consequence concrete (taking $k = 3$, and a worst-case atom constant $|V(A)| \approx (wk)^2$):

Graph class	width w	peak atoms $O((wk)^2)$	consequence
trees, k -degenerate cores emptied	1	$O(k^2) \approx 9$	any n , trivially
series-parallel / outerplanar	≤ 2	≈ 36	any n
bounded treewidth (fixed w)	$O(1)$	$O(k^2)$ const.	any n at constant device
“sparse with small dense pockets”	small const. blocks	const.	any n
planar (general)	$\Theta(\sqrt{n})$	$\Theta(nk^2)$	linear in n (was quadratic)
arbitrary dense	$\Theta(n)$	$\Theta((nk)^2)$	no improvement

Table 1: Peak device size is set by the decomposition width w , not by n . For bounded-width classes the device size is constant in n .

Corollary 7.2 (Hundreds of vertices, concretely). *A graph whose pieces all have $\leq w$ vertices is solvable with peak device size $O((wk)^2)$ regardless of n . For instance, with $w = 15$ and $k = 4$ the peak is $\approx (60)^2 = 3600$ logical atoms before gadgetization and a few hundred atoms after a crossing-minimized layout of each small piece — well within a 1000-atom array — so a graph of several hundred vertices is colored as a sequence of small device solves. With $w = O(1)$ (e.g. a chain of small blocks) the peak is a fixed handful of atoms and n is unbounded; see Example 8.2.*

Remark 7.3 (Cost honesty). The improvement is real but not free. (i) For non-clique separators the $O(k^b)$ factor is exponential in the separator size b , so the method is practical only when separators are small (clique separators give b effectively 0; bounded treewidth gives $b \leq w + 1 = O(1)$). (ii) Finding a good decomposition is itself work: clique-separator atoms are computable in polynomial time [5], and an optimal-width tree decomposition is fixed-parameter

tractable [2] with good heuristics in practice. (iii) Dense graphs have $w = \Theta(n)$ and gain nothing — decomposition helps exactly the structured, sparse instances, which is where the rest of the pipeline already lives.

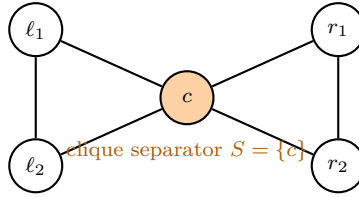
Remark 7.4 (Toward the type system). Decomposition turns compilation into a *composition of certified sub-solves*. Each piece carries a coloring decoded against the instance that produced it, and the boundary precoloring is a constraint shared between adjacent pieces. A future type system can track this directly: provenance of each partial coloring, the `precolor` obligations linking neighbors, and a stitching rule whose well-typedness witnesses Theorem 6.1. “Reaching hundreds of vertices” and “what does the type system buy us” become the same question — compositional, certificate-carrying compilation.

8 Worked examples

We finish with fully explicit examples, in the step-by-step style of the companion examples document.

8.1 Example A — the bowtie: a single cut vertex

The graph. Two triangles sharing one vertex c (the *bowtie*): $\{\ell_1, \ell_2, c\}$ and $\{c, r_1, r_2\}$. Here $S = \{c\}$ is a clique separator of size 1 (a cut vertex), and the two pieces are the two triangles. $n = 5$.



Step 1 — decompose. Cut at c : piece $P_1 = \{\ell_1, \ell_2, c\}$ (a triangle), piece $P_2 = \{c, r_1, r_2\}$ (a triangle). Width $w = 3$. Each triangle needs $k = 3$ (Thm. 3.1 gives $\chi = \max(3, 3) = 3$).

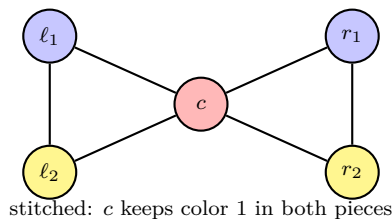
Step 2 — solve P_1 (no boundary yet). Build the color-choice graph of the triangle with $k = 3$ and solve, exactly as in the companion document: the MWIS picks one slot per vertex in distinct rows, e.g.

$$c \mapsto 1, \quad \ell_1 \mapsto 2, \quad \ell_2 \mapsto 3.$$

Record this. Note the device only ever sees a 3-vertex piece ($|V(G'_1)| = nk = 9$ logical atoms), not the whole bowtie.

Step 3 — solve P_2 with boundary precolored. The boundary is $B = \{c\}$ with $\beta(c) = 1$ (inherited from P_1). By Definition 4.1, c keeps only its slot $(c, 1)$; r_1, r_2 keep all three slots. Solving $G'_2(\beta)$ (Prop. 4.2) yields a proper extension, e.g.

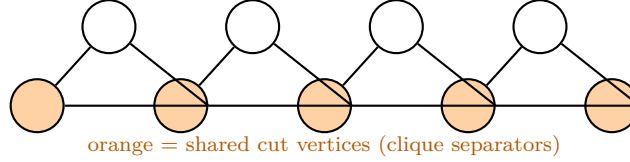
$$c \mapsto 1 \text{ (fixed)}, \quad r_1 \mapsto 2, \quad r_2 \mapsto 3.$$



Step 4 — stitch. Both pieces agree on c (color 1) by construction, so their union is a proper 3-coloring of the bowtie (Thm. 6.1). **Peak device size:** one triangle (9 logical atoms), versus 15 for the monolithic bowtie — and crucially the peak is independent of how many triangles we attach, which is the next example.

8.2 Example B — a chain of triangles: $n \rightarrow \infty$ at constant device size

The graph. Glue t triangles in a line, each sharing one cut vertex with the next: vertices $v_0, v_1, \dots, v_{2t}$, with triangles $\{v_{2j}, v_{2j+1}, v_{2j+2}\}$ for $j = 0, \dots, t-1$. Then $n = 2t + 1$ grows without bound, but every piece is a triangle, so $w = 3$ for all t .

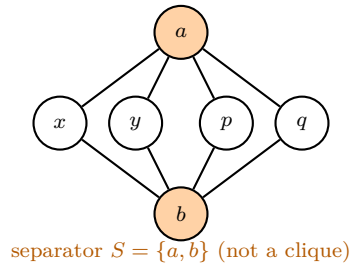


Solve. Process triangles left to right. The first is solved freely (3 colors). Each subsequent triangle shares exactly one already-colored vertex; precolor it (boundary size 1) and solve, then realign by the color permutation of Theorem 3.1 if needed. Every device solve is a single triangle: $|V(G'_i(\beta))| \leq 9$ logical atoms.

The point. By Theorem 7.1, the peak device size is $O((3k)^2)$ — a fixed handful of atoms — for every t . A 201-vertex chain ($t = 100$) is colored by 100 tiny sequential solves; a 2001-vertex chain the same way. The atom wall has been removed for this class entirely: n is unbounded at constant hardware size. This is the concrete sense in which hundreds (or thousands) of vertices are reachable.

8.3 Example C — a non-clique separator: paying the k^b price

The graph. Two squares glued along a non-adjacent vertex pair. Let P_1 be the 4-cycle $axby$ and P_2 the 4-cycle $apbq$, sharing $S = \{a, b\}$, with a, b not adjacent. Now S is a separator but not a clique, so the free permutation trick does not apply — we must enumerate boundary colorings.



Solve with $k = 2$. The boundary table over $S = \{a, b\}$ has at most $k^{|S|} = 2^2 = 4$ candidates: $(a, b) \in \{(1, 1), (1, 2), (2, 1), (2, 2)\}$. For each, run the precolored solve on each square:

- $\beta = (a=1, b=2)$: each square completes $(x, p \mapsto \dots)$, compatible on both sides — a global 2-coloring exists. ✓
- $\beta = (a=1, b=1)$: in P_1 , vertex x is adjacent to both a and b , now both colored 1, so x needs a second color — fine for $k = 2$ ($x \mapsto 2$); but y likewise must be 2, and a, b equal is allowed since a, b non-adjacent. This β also completes. ✓

We keep any β that completes on *both* pieces and stitch. The extra cost versus the clique case is the up-to-4 boundary colorings we had to try — exactly the $O(k^b)$ factor of Theorem 7.1 with $b = |S| = 2$. Had S been a clique (an edge $a-b$), only the β with $a \neq b$ would survive and no enumeration would be needed.

Lesson. Small separators keep the k^b factor tiny; clique separators make it vanish. This is why the front-end prefers clique-separator atoms and, failing that, low-treewidth bags — precisely the structure that also keeps the per-piece encoding small.

References

- [1] Stefan Arnborg and Andrzej Proskurowski. Linear time algorithms for NP-hard problems restricted to partial k -trees. *Discrete Applied Mathematics*, 23(1):11–24, 1989.
- [2] Hans L Bodlaender. A linear-time algorithm for finding tree-decompositions of small treewidth. *SIAM Journal on Computing*, 25(6):1305–1317, 1996.
- [3] Gabriel Andrew Dirac. On rigid circuit graphs. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 25(1–2):71–76, 1961.
- [4] Neil Robertson and Paul D Seymour. Graph minors. II. algorithmic aspects of tree-width. *Journal of Algorithms*, 7(3):309–322, 1986.
- [5] Robert E Tarjan. Decomposition by clique separators. *Discrete Mathematics*, 55(2):221–232, 1985.